

For the threads-based implementation, the most straightforward critical point is limiting how many animals can eat simultaneously. At most B cats may occupy bowls at once, and separately, at most B mice may do the same. Since these are independent capacity limits, we modeled each with its own counting semaphore: `cat_bowls` and `mouse_bowls`, both initialized to B. A cat calls `sem_wait(cat_bowls)` before sitting at a bowl and `sem_post(cat_bowls)` when it leaves, and mice do the same on `mouse_bowls`. The semaphore's counting behavior naturally enforces the capacity constraint: when B cats are already eating, the next hungry cat blocks until one finishes and posts. We chose semaphores over mutexes here because a mutex only allows one holder at a time, whereas the scenario allows up to B concurrent feeders of the same type.

The more complex requirement is the global "cats present" signal. Mice must not begin eating if even one cat is currently feeding, and any mouse already eating must flee the instant a cat arrives. A semaphore alone cannot express this two-sided condition, because there is no built-in way to broadcast that a new state has been reached to an arbitrary number of waiting threads. We therefore used an integer counter `active_cats` protected by a mutex `cat_count_mutex`, paired with two condition variables. The first, `no_cats`, is broadcast when `active_cats` drops from 1 to 0, releasing any mice that were waiting to enter. The second, `cats_present`, is broadcast when `active_cats` rises from 0 to 1, signaling any feeding mice that they must flee. The cat thread acquires the mutex, updates the counter, broadcasts the appropriate signal, and immediately releases the mutex. Crucially, the mutex is never held while the cat is actually eating only during the brief counter update. Holding it across the sleep would prevent mice from ever reading the counter, effectively deadlocking the simulation.

A subtle but important race exists between a mouse confirming that no cats are present and actually acquiring a bowl. A mouse might check `active_cats == 0`, find it true, and then be preempted before calling `sem_wait`. In that window, a cat can arrive and increment the counter, yet the mouse proceeds to acquire a bowl and begins eating in violation of the rules. To close this window, after acquiring the bowl the mouse re-checks `active_cats` under the mutex before declaring itself to be feeding. If a cat has arrived in the gap, the mouse immediately releases the bowl with `sem_post` and loops back to waiting. This double-check pattern is the standard response to time-of-check / time-of-use races and ensures that no mouse ever eats while a cat is present and that no bowl slot is leaked in the process.

Once a mouse is eating, it must be able to react to a cat arriving at any moment. The naive approach of sleeping for a fixed feeding duration would cause the mouse to finish its sleep before noticing the cat, violating the rule that mice must flee immediately. Our solution is to feed in short repeated bursts of 50 ms. After each burst the mouse acquires the mutex and inspects `active_cats` — if a cat has arrived, it sets a fled flag, releases the bowl, and exits the feeding loop. The burst length controls the trade-off between responsiveness and CPU overhead; at 50 ms, a cat arrival is visible to a feeding mouse within at most one burst period, which is acceptable for simulation purposes. A more elegant alternative would be `pthread_cond_timedwait` on the `cats_present` condition variable, which would let the mouse truly sleep and wake only on a cat signal rather than polling. We chose the burst approach for its

simplicity and ease of debugging, but the timed wait variant would be preferable in a production system.

Finally, a practical synchronization point is console output. With many threads printing timestamped events concurrently, unsynchronized `printf` calls produce interleaved and unreadable lines. We protect every call to `sim_log` with a dedicated `log_mutex`, held only for the duration of the format-and-print operation. This introduces negligible contention while guaranteeing each log line appears as a single atomic unit.

The overarching design principle across all of these points is that no mutex is ever held while a thread is sleeping or eating. This ensures that a slow or long-feeding thread cannot block others from reading or updating shared state, eliminating the most common source of deadlock and priority inversion in producer-consumer style simulations.